

PROJECT #0 - C++ PRIMER

Last Updated: Aug 26, 2025

⚠ Do not post your project on a public Github repository.

Overview

All the programming projects this semester will be written on the [Bustub](#) database management system. This system is written in C++. To make sure that you have the necessary C++ background, you must complete a simple programming assignment to assess your knowledge of basic C++ features. You will not be given a grade for this project, but you **must complete the project with a perfect score** before being allowed to proceed in the course. Any student unable to complete this assignment before the deadline will be asked to drop the course.

All of the code in this programming assignment must be written in C++. The projects will be specifically written for C++17, but we have found that it is generally sufficient to know C++11. If you have not used C++ before, here are some resources to help:

- [15-445 Bootcamp](#), which contains several small examples to get you familiar with C++11 features.
- [LearnCPP](#) is a useful resource that includes quizzes to test your knowledge.
- [cppreference](#) has more detailed documentation of language internals.
- [A Tour of C++](#) and [Effective Modern C++](#) are also digitally available from the CMU library.

If you are using VSCode, we recommend you to install [CMake Tools](#), [C/C++ Extension Pack](#) and [clangd](#). After that, follow this tutorial to learn how to use the visual debugger in VSCode: [Debug a C++ project in VS Code](#).

If you are using CLion, we recommend you to follow this tutorial: [CLion Debugger Fundamentals](#).

If you prefer to use [gdb](#) for debugging, there are many tutorials available to teach you how to use [gdb](#). Here are some that we have found useful:

- [Debugging Under Unix: gdb Tutorial](#)
- [GDB Tutorial: Advanced Debugging Tips For C/C++ Programmers](#)
- [Give me 15 minutes & I'll change your view of GDB](#) [VIDEO]

This is a single-person project that will be completed individually (i.e. no groups).

- **Release Date:** Aug 25, 2025
- **Due Date:** Sep 07, 2025 @ 11:59pm

Project Specification

Consider the following scenario: you are the administrator of a popular blog website, and you've been receiving reports on certain accounting spamming excessively. To map out overall network usage and detect potential DDoS attacks, you want a real-time approach to count how often each IP address appears in the incoming request stream. However, the stream is huge, which makes traditional data structures either too slow or too memory-hungry. This is where the **Count-Min Sketch** comes in!

[Count-min sketch](#) (CM Sketch) is a [probabilistic](#) data structure that approximates frequency counts of items in a stream using sublinear memory. It maintains a compact 2-dimensional array of counters addressed by d independently seeded hash functions. Each update increments one cell per row, and a query returns the minimum of those counters. Moreover, count-min sketch is mergeable, meaning that the sum of two sketches is equivalent to constructing a single sketch over the concatenation of the corresponding input streams. Count-min sketch is widely used for network traffic monitoring, streaming analytics, and database system optimization.

Count-min sketch is based on the following parameters:

- **width (w)** – Number of columns in the hash matrix; each hash maps an item to an index in $[0, w-1]$. Larger $w \Rightarrow$ smaller additive error.
- **depth (d)** – Number of rows / independent hash functions. Larger $d \Rightarrow$ lower probability of a bad overestimate.
- **hash family / seeds** – A way to derive d pairwise-independent hash functions (e.g., by seeding a base hash differently for each row).

To explain how this data structure functions, let follow the example at the beginning and consider the following input stream:

`["24.156.99.202", "172.217.22.14", "64.104.78.227", "24.156.99.202"]`. Let's use a count-min sketch with width 4 and depth 3. For each of the 3 rows, hash the string (using that row's seed) to produce an integer, reduce it modulo 4 to get a column index, and increment the counter at (row, column).

First, initialize the hash matrix (3 rows $\times$ 4 columns) with all zeros:

```
0 0 0 0
0 0 0 0
0 0 0 0
```

Now process the stream. We first insert "24.156.99.202". Assume the following hash positions (mod 4):

- hash1 $\rightarrow$ 2
- hash2 $\rightarrow$ 0
- hash3 $\rightarrow$ 3

We update the cells accordingly:

```
0 0 1 0
1 0 0 0
0 0 0 1
```

Next, we insert "172.217.22.14" and increment counters at the hash positions below:

- hash1 $\rightarrow$ 1
- hash2 $\rightarrow$ 0 (collision with the first item in row 1)
- hash3 $\rightarrow$ 2

Update:

```
0 1 1 0
2 0 0 0
0 0 1 1
```

Now we insert "64.104.78.227":

- hash1 $\rightarrow$ 3
- hash2 $\rightarrow$ 1
- hash3 $\rightarrow$ 2 (collision with the second item in row 3)

The table becomes:

```
0 1 1 1
2 1 0 0
0 0 2 1
```

Finally, repeat "24.156.99.202" (same hash positions as before: 0, 0, 3). Increment those cells again:

Finally, repeat: 24.156.99.202. (Same hash positions as before: 2, 0, 3). Increment those cells again:

```
0 1 2 1
3 1 0 0
0 0 2 2
```

Now, let's estimate the frequency of "24.156.99.202". Let's look up the same columns used when inserting that key:

- Row 0, col 2 → **2**
- Row 1, col 0 → **3**
- Row 2, col 3 → **2**

The estimate is the minimum across rows:

```
Estimate("24.156.99.202") = min(2, 3, 2) = 2
```

Why take the minimum? Each row's counter can be inflated by collisions with other items, but the minimum across independent hash rows gives the tightest upper bound on the true frequency.

Resources

To understand more about Count-min sketch and why they work,

- A [visual walkthrough](#) illustrating the inner workings of the count-min sketch.
- A [comprehensive overview](#) of count-min sketch and its practical applications (the full video also covers Bloom Filter and HyperLogLog, two other widely applied data structures for large-scale data processing).
- A concise [paper](#) analyzing the time and space bounds of the data structure.

Instructions

You will have to complete the following task for this project:

Task #1

Implement a basic Count-min sktech data structure supporting insertion, count estimation, and merging.

In `count_min_sketch.h`, following functions are to be implemented:

- `CountMinSketch(width, depth)`: constructor, creates a count-min sketch with `width` columns (buckets) and `depth` rows (hash functions).
- `CountMinSketch(&&other)`: move constructor, transfers ownership of sketch resources from another instance.
- `operator=(&&other)`: move assignment, moves sketch resources from another instance to this one.
- `Insert(item)`: inserts the specified item into the count-min sketch. **this is expected to be thread-safe**
- `Count(&item)`: returns the estimated frequency of the item.
- `Clear()`: resets the data structure from previous streams.
- `Merge(&other)`: creates a new sketch by combining counter values from two compatible sketches.
- `TopK(k, &candidates)`: practical usage of count-min sketch--given the `candidates` that have been stored in the count-min sktech, return the `k` candidates with the most estimated counts.

Feel free to design your helper functions to assist with the implementation.

Important Information

- For constructing hash functions for the matrix, please use the seeded hash function "`common/util/hash_util.h`" from the bustub repository. **Please refrain from using hash functions from external libraries or implementing your own, since this might influence whether you pass the test suite we provide!**
- The test suite includes parallel tests. However, **we ONLY expect thread-safe implementation for `Insert(item)`**. In other words, your implementation must correctly handle scenarios where multiple threads simultaneously perform insertions into multiple count-min sketches.
- You may notice the last test compares the performance of your implementation for `Insert(item)` against one that is strictly sequential. **You could only pass this test if the relative speedup of your implementation is larger than 1.2. We expect you NOT to use only a single global latch to guard the whole data structure.** If you do so, the contention ratio will be effectively around 1.0. There are many ways to do this. As a hint, try thinking of ways to break down the latch granularity or, even better, not to use a latch at all (you may find [compare_exchange](#) helpful if you aim for the latter). If you find this difficult to reason about, try passing other tests with a global latch first before attempting to optimize for this one.

Creating Your Own Project Repository

If the below `git` concepts (e.g., repository, merge, pull, fork) do not make sense to you, please spend some time [learning git](#) first.

Follow the [instructions](#) to setup your own PRIVATE repository and your own development branch. If you have previuosly forked the repository through the GitHub UI (by clicking Fork), PLEASE DO NOT PUSH ANY CODE TO YOUR PUBLIC FORKED REPOSITORY! Make sure your repository is PRIVATE before you `git push` any of your code.

If the instructor makes any changes to the code, you can merge the changes to your code by keeping your private repository connected to the CMU-DB master repository. Execute the following commands to add a remote source:

```
$ git remote add public https://github.com/cmu-db/bustub.git
```

You can then pull down the latest changes as needed during the semester:

```
$ git fetch public
$ git merge public/master
```

Setting Up Your Development Environment

First install the packages that BusTub requires:

```
# Linux
$ sudo build_support/packages.sh
# macOS
$ build_support/packages.sh
```

See the [README](#) for additional information on how to setup different OS environments.

To build the system from the commandline, execute the following commands:

```
$ mkdir build
$ cd build
$ cmake -DCMAKE_BUILD_TYPE=Debug ..
$ make -j`nproc`
```

We recommend always configuring CMake in debug mode. This will enable you to output debug messages and check for memory leaks (more on this in below sections).

Testing

You can test the individual components of this assignment using our testing framework. We use `GTest` for unit test cases. You can disable tests in GTest by adding a `DISABLED_` prefix to the test name. To run the tests from the command-line:

```
$ cd build
```

```
> cd build
$ make -j$(nproc) count_min_sketch_test
$ ./test/count_min_sketch_test
```

In this project, there are no hidden tests. In the future, the provided tests in the starter code are only a subset of the all the tests that we will use to evaluate and grade your project. You should write additional test cases on your own to check the complete functionality of your implementation.

? Make sure that you remove the `DISABLED_` prefix from the test names otherwise they will **not** run!

Formatting

Your code must follow the [Google C++ Style Guide](#). We use [Clang](#) to automatically check the quality of your source code. Your project grade will be **zero** if your submission fails any of these checks.

Execute the following commands to check your syntax. The `format` target will automatically correct your code. The `check-llint` and `check-clang-tidy` targets will print errors that you must manually fix to conform to our style guide.

```
$ make format
$ make check-clang-tidy-p0
```

Memory Leaks

For this project, we use [LLVM Address Sanitizer \(ASAN\)](#) and [Leak Sanitizer \(LSAN\)](#) to check for memory errors. To enable ASAN and LSAN, configure CMake in debug mode and run tests as you normally would. If there is memory error, you will see a memory error report. Note that macOS **only supports address sanitizer without leak sanitizer**.

In some cases, address sanitizer might affect the usability of the debugger. In this case, you might need to disable all sanitizers by configuring the CMake project with:

```
$ cmake -DCMAKE_BUILD_TYPE=Debug -DBUSTUB_SANITIZER= ..
```

Development Hints

You can use `BUSTUB_ASSERT` for assertions in debug mode. Note that the statements within `BUSTUB_ASSERT` will NOT be executed in release mode. If you have something to assert in all cases, use `BUSTUB_ENSURE` instead.

We will test your implementation in release mode. To compile your solution in release mode,

```
$ mkdir build_rel && cd build_rel
$ cmake -DCMAKE_BUILD_TYPE=Release ..
```

? Post all of your questions about this project on Piazza. Do **not** email the TAs directly with questions.

? TAs will **not** look into your code or help you debug in this project.

Grading Rubric

In order to pass this project, you must ensure your code follows the following guidelines:

1. Does the submission successfully execute all of the test cases and produce the correct answer?
2. Does the submission execute without any memory leaks?
3. Does the submission follow the code formatting and style policies?

Note that we will use additional test cases to grade your submission that are more complex than the sample test cases that we provide you in future projects.

Late Policy

There are no late days for this project.

Submission

You will submit your implementation to Gradescope:

- <https://www.gradescope.com/courses/1074751>

Run this command in `build` directory and it will create a `zip` archive called `project0-submission.zip` that you can submit to Gradescope.

```
$ make submit-p0
```

Although you are allowed submit your answers as many times as you like, you should **not** treat Gradescope as your only debugging tool. Most students submit their projects near the deadline, and thus Gradescope will take longer to process the requests. You may not get feedback in a timely manner to help you debug problems. Furthermore, the output from Gradescope is unlikely to be as informative as the output from a debugger (like `gdb`), provided you invest some time in learning to use it.

? CMU students should use the Gradescope course code announced on Piazza.

Collaboration Policy

- Every student must work individually on this assignment.
- Students are allowed to discuss high-level details about the project with others.
- Students are **not** allowed to copy the contents of a white-board after a group meeting with other students.
- Students are **not** allowed to copy solutions from another person.
- Students are allowed to use generative AI to help with development but they are ultimately responsible for their work.

! **WARNING:** All of the code for this project must be your own. You may not copy source code from other students or other sources that you find on the web. Plagiarism **will not** be tolerated. See CMU's [Policy on Academic Integrity](#) for additional information.